#1

CS 0447
Introduction to
Computer Programming

Binary, Hex, Octal

Luís Oliveira

Original slides by: Jarrett Billingsley
Modified with bits from: Bruce Childers, David Wilkinson

Fall 2020

What do I need to know now!

<u>The classes will be recorded!</u>

- You will be able to access the videos online
  - They are for your personal use only!
  - Do not distribute them!

- You don't need to turn on your camera
  - If you do, you may be recorded

- You can ask questions via text!
  - Chat is great for that. If I don't stop and read your questions, ask them again
  - But feel free to interrupt me at any point.

Class Announcements

Recitation are happening!!

Summary

- Numbers and bases
  - Why is 2020 worth 2020??
  - Or is it?
    - Maybe it's worth 8224?

- Number representation
  - Binary – what do 0s and 1s mean?

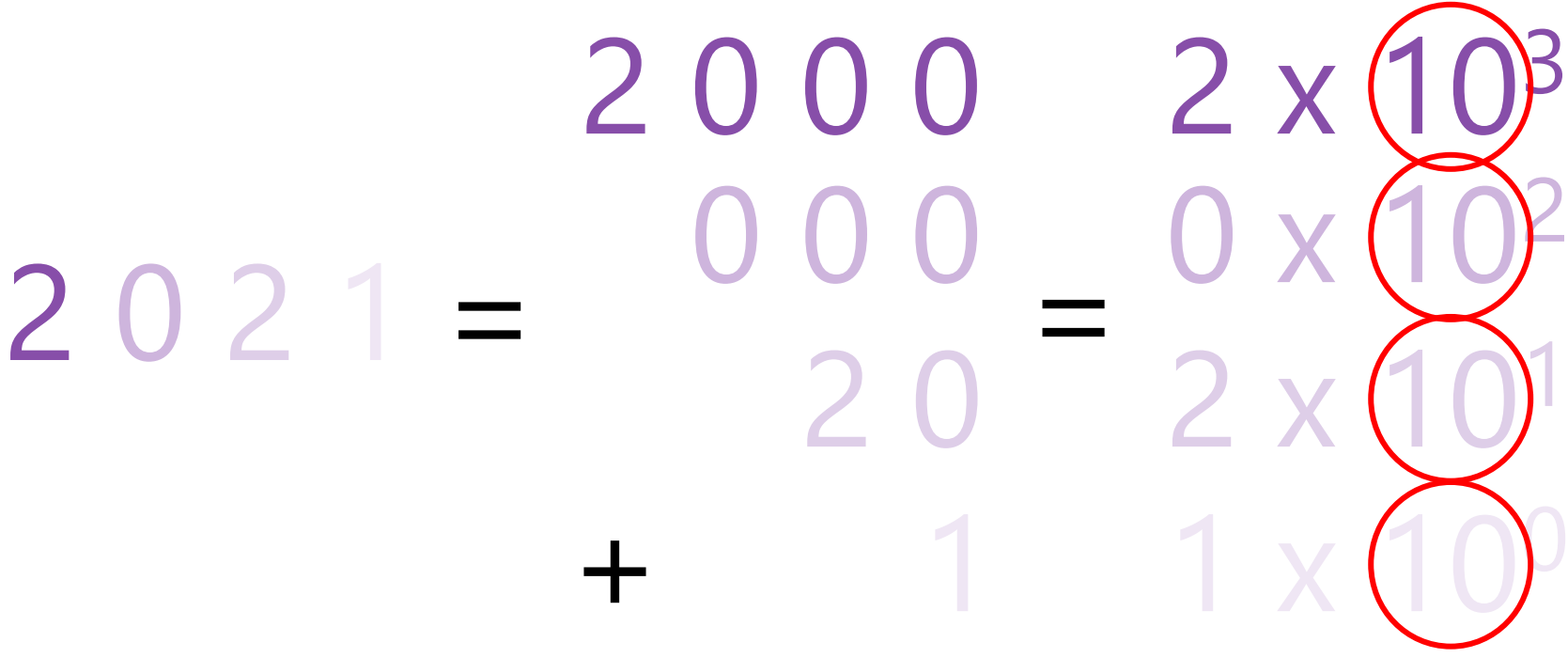
- Other representations
  - Hexadecimal and octal

- How does a computer interpret them?
  - (bait) The answer will surprise you!!

Numbers and bases

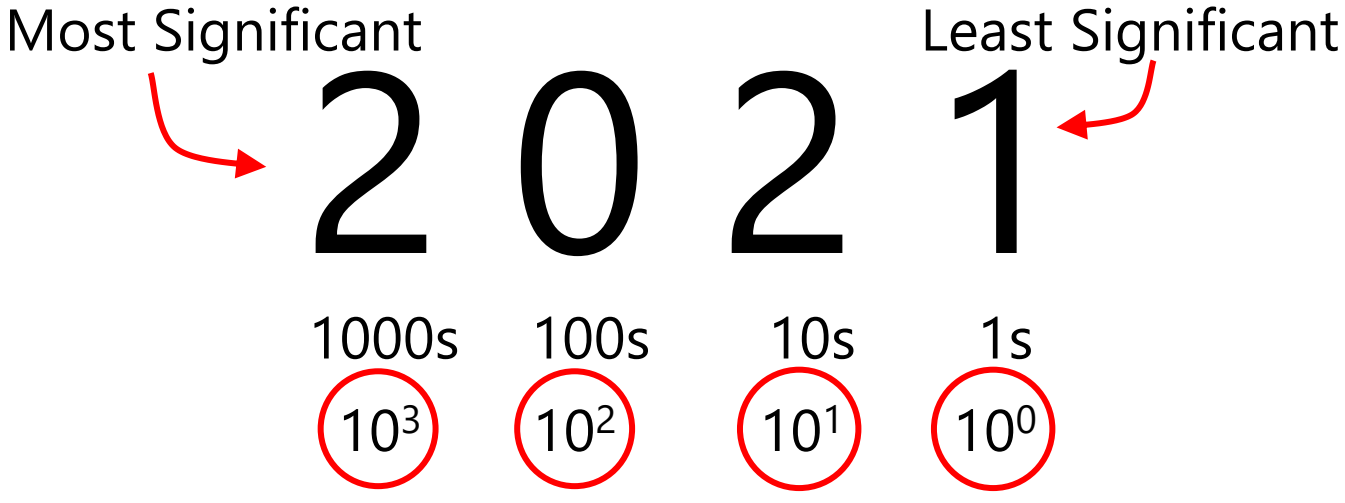
Positional number systems

- The numbers we use are written **positionally**: the position of a digit within the number has a meaning.



Positional number systems

- The numbers we use are written **positionally**: the position of a digit within the number has a meaning.



- How many (digits) **symbols** do we have in our number system?
  - 10: 0, 1, 2, 3, 4, 5 ,6 ,7, 8, 9

Range of numbers

Suppose we have a 4-digit numeric display.



- What is the smallest number it can show?
- What is the biggest number it can show?
- How many *different* numbers can it show?
  - 9999 - 0 + 1 = 10,000
  - What power of 10 is 10,000?
    - 10^4

So… with the numbers we use every day…

- A number represented by the digits

$$d_n d_{n-1} \dots d_1 d_0$$

- Has the value

$$d_n \times 10^n + d_{n-1} \times 10^{n-1} + \cdots + d_1 \times 10^1 + d_0 \times 10^0$$

- Using n digits we can represent 10^n different numbers
- The smallest non-negative number representable with n digits is 0
- The largest number representable with n digits is $10^n - 1$
- Using 10 symbols: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Numeric Bases

- These 10s keep popping up… and for good reason
- We use a **base-10 (decimal)** numbering system
  - 10 different **symbols**, and each place is a power of 10
- But we can use (almost) any number as a base!
- The most common bases when dealing with computers are **base-2 (binary)**, **base-16 (hexadecimal)**, and (rarely) **base-8 (octal)**
- When dealing with multiple bases, you can write the base as a subscript to be explicit about it:

$$5_{10} = 101_2$$

Making a number system

Using base **B**

- A number represented by the digits

$$d_n d_{n-1} \ldots d_1 d_0$$

- Has the value

$$d_n \times \mathbf{B}^n + d_{n-1} \times \mathbf{B}^{n-1} + \cdots + d_1 \times \mathbf{B}^1 + d_0 \times \mathbf{B}^0$$

- Using n digits we can represent $\mathbf{B}^n$ different numbers
- The smallest non-negative number representable with n digits is 0
- The largest number representable with n digits is $\mathbf{B}^n - 1$
- Using **B** symbols



If you use base 0 you don't need to remember any symbols

Binary – Base 2

How many symbols in binary????

2

Let's make a base-2 number system

Using base **2**

- A number represented by the digits

$$d_n d_{n-1} \dots d_1 d_0$$

- Has the value

$$d_n \times \mathbf{2}^n + d_{n-1} \times \mathbf{2}^{n-1} + \cdots + d_1 \times \mathbf{2}^1 + d_0 \times \mathbf{2}^0$$

- Using n digits we can represent $\mathbf{2}^n$ different numbers
- The smallest non-negative number representable with n digits is 0
- The largest number representable with n digits is $\mathbf{2}^n - 1$
- Using **2** symbols: 0, 1

Binary (base-2)

- We call a **B**inary dig**IT** a **bit** – a single 1 or 0
- When we say an *n*-bit number, we mean one with *n* binary digits

MSB LSB

1001 0110 =

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128s	64s	32s	16s	8s	4s	2s	1s

$1 \times 128 +$
$0 \times 64 +$
$0 \times 32 +$
$1 \times 16 +$
$0 \times 8 +$
$1 \times 4 +$
$1 \times 2 +$
0×1

$= 150_{10}$

To convert binary to decimal: ignore 0s, add up place values wherever you see a 1.

It's the only odd number!

Making change

- You want to give someone $9.63 in change, using the fewest bills and coins possible. How do you count it out?

	$5×1		$1×4		25¢×2		10¢×1		5¢×0		1¢×3	
Left:	$9.63	-$5=	$4.63	-$4=	$0.63	-50¢=	$0.13	-10¢=	$0.03	-0¢=	$0.03	-3¢=	$0.00













- Biggest to smallest
- Most significant to least significant
- WHERE COULD THIS BE GOING...

Converting decimal to binary

- You want to convert the number 83_{10} to binary.

	128s	64s	32s	16s	8s	4s	2s	1s
Left:	83 - 0 =	83 - 64 =	19 - 0 =	19 - 16 =	3 - 0 =	3 - 0 =	3 - 2 =	1 - 1 = 0

01010011

- For each place from **MSB**:
- If place value < remainder:
  - digit = 1
  - remainder = remainder - place
- Else, digit = 0.

Bits, bytes, nibbles, and words

- A ***bit*** is one binary digit, and its unit is **lowercase** b.

- A ***byte*** is an 8-bit value, and its unit is **UPPERCASE** B.
  - This is (partially) why your 30 mega*bit* (Mbps) internet connection can only give you at most 3.57 mega*bytes* (MB) per second!

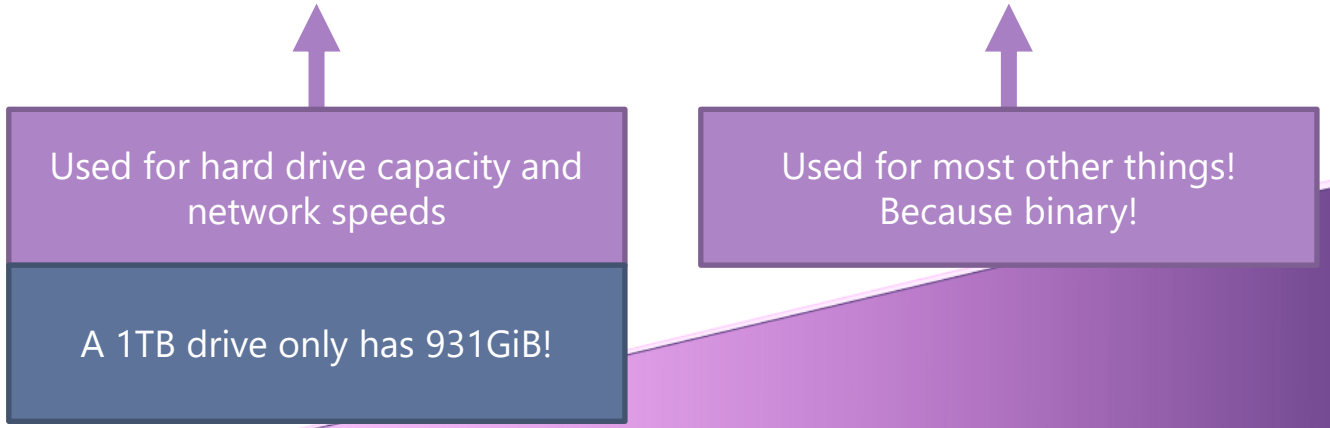
- A ***nibble*** (also nybble) is 4 bits – half of a byte
  - Corresponds nicely to a single hex digit.

- A ***word*** is the "most comfortable size" of number for a CPU.
- When we say "32-bit CPU," we mean its ***word*** size is 32 bits.
  - This means it can, for example, add two 32-bit numbers at once.

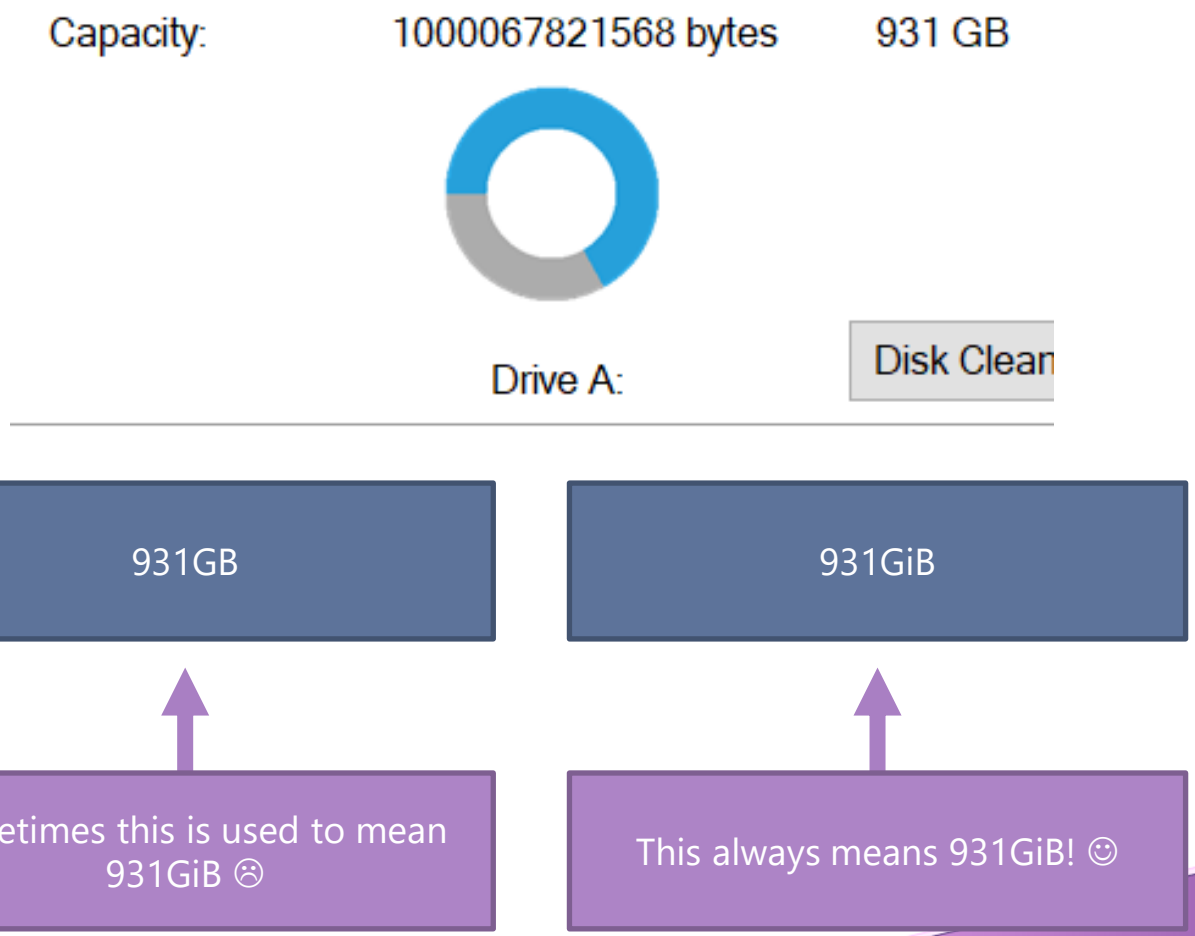
- **BUT WATCH OUT:**
  - Some things (Windows, x86) use **word** to mean **16 bits** and **double word** (or **dword**) to mean **32 bits.**

Kilo, mega, tera

Potatoes	Bytes	Bytes
1g (gram)	1B (Byte)	1B (Byte)
1kg (Kilogram) = 1000g	1kB (Kilobyte) = 1000B	1kiB (Kibibyte) = 1024B (power of 2 nearest to 1000)
1Mg (Megagram) = 1000Kg	1MB (Megabyte) = 1000kB	1MiB (Mebibyte) = 1024kiB
1Gg (Gigagram) = 1000Mg	1GB (Gigabyte) = 1000MB	1GiB (Gibibyte) = 1024MiB
1Tg (Teragram) = 1000Gg	1TB (Terabyte) = 1000GB	1TiB (Tebibyte) = 1024GiB
1Eg (Exagram) = 1000Tg	1EB (Exabyte) = 1000TB	1EiB (Exbibyte) = 1024TiB



The real world IS CONFUSING!!!!



Why binary? *Whynary?*

- Why indeed?
  - $\log_2(10) = 3.322$
  - The number of bits required to represent 10 digits
  - The increase on number of components in a computer!

- Because it's soooooo much easier to implement the hardware!
  - And robust!

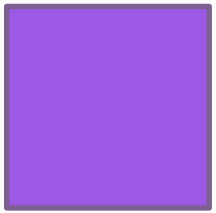
- Arithmetic becomes really easy (as we'll see in several weeks)

Why binary? *Whynary?*

- Why indeed?



- What color is this?

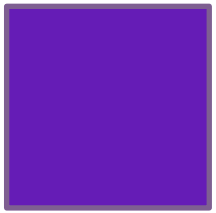


Why binary? *Whynary?*

- Why indeed?



- What color is this?



Why binary? *Whynary?*

- Why indeed?
  - $\log_2(10) = 3.322$
  - The number of bits required to represent 10 digits
  - The increase on number of components in a computer!

- Because it's soooooo much easier to implement the hardware!
  - And robust!

- Arithmetic becomes really easy (as we'll see in several weeks)

Everything in a computer is a number

- So, everything on a computer is represented in binary.
  - **everything.**
    01100101 01110110 01100101 01110010 01111001 01110100 01101000 01101001 01101110 01100111 00001010 00000000

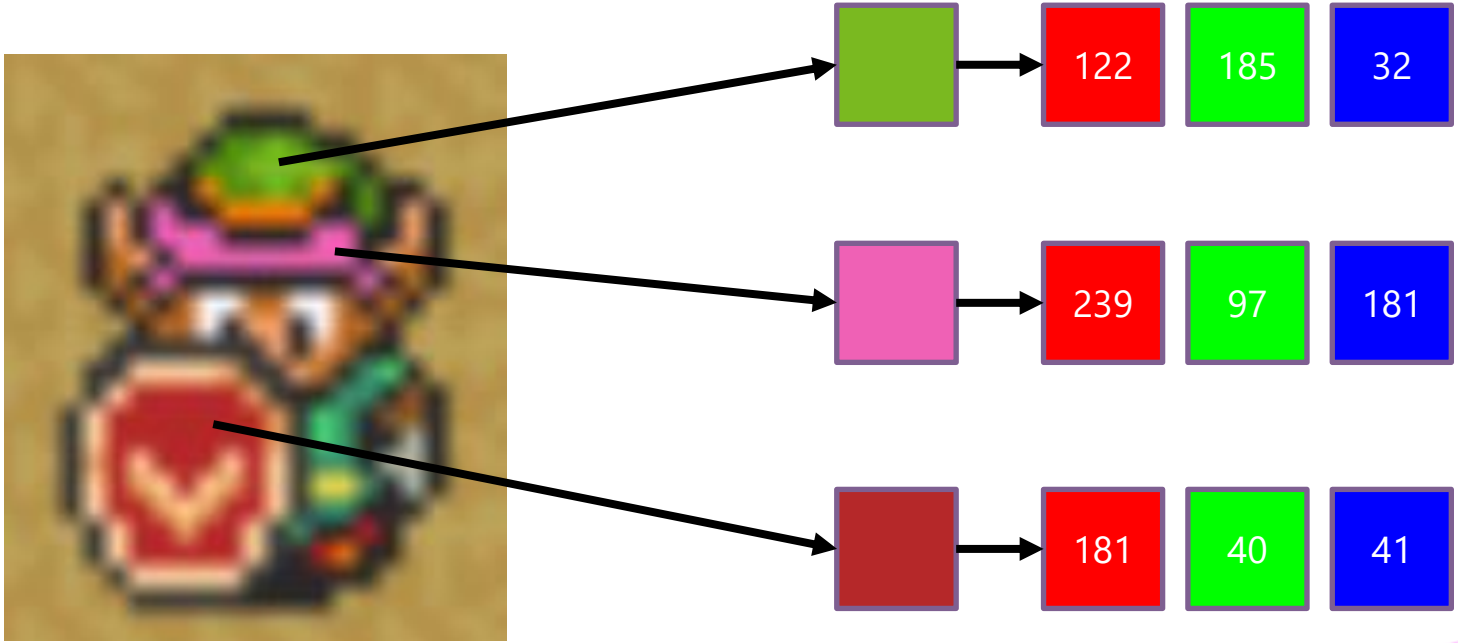
- Java strings are encoded using **UTF-16**
  - Most letters and numbers in the English alphabet are < 128.
  - "Strings are numbers"
    - **83 116 114 105 110 103 115 32 97 114 101 32 110 117 109 98 101 114 115 0**

- **ASCII** is also pretty common (the best kind of common)
  - That's what we will be using → 8 bit numbers represent characters
  - Letters and numbers (and most/all ascii characters) have the same value as UTF-16

Do try this at home: what does this mean?

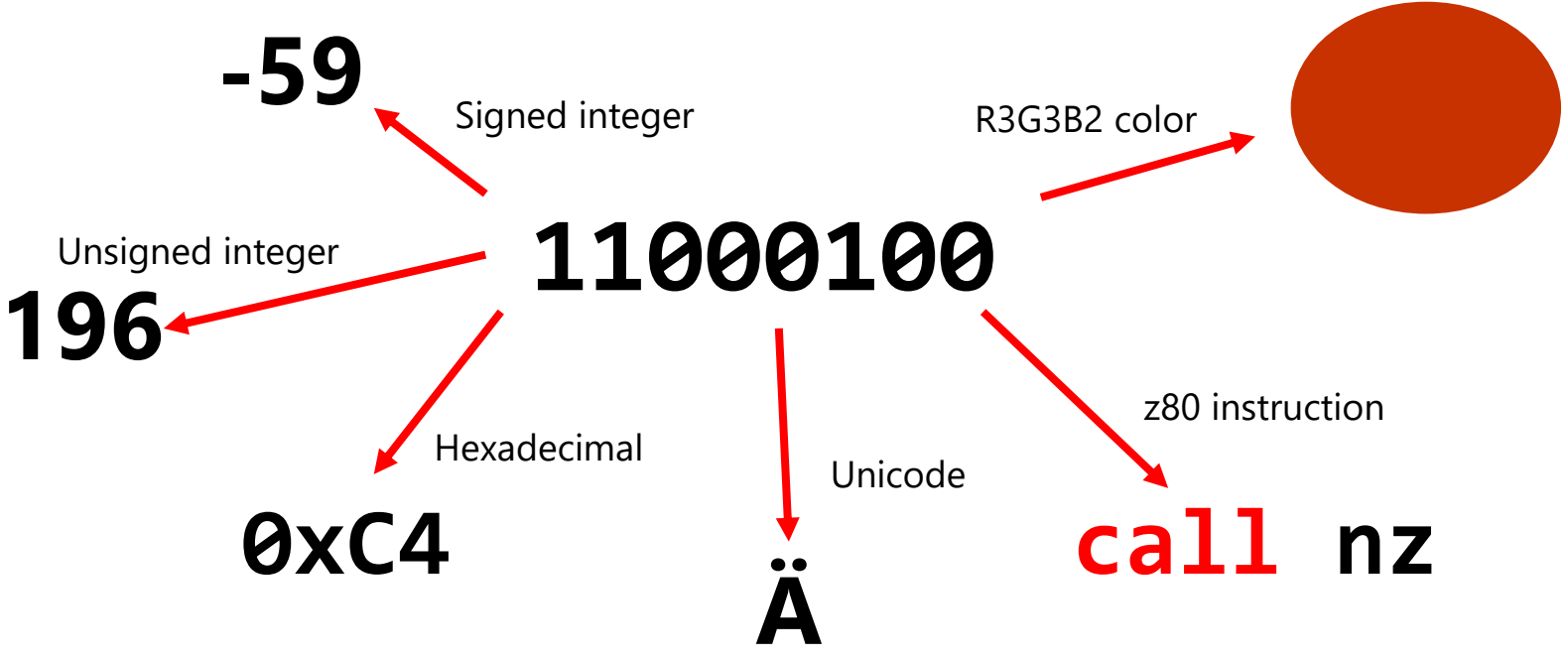
- 71 111 111 100 32 74 111 98 0

- Images and colors? Numbers!
- Videos? Numbers!



One number, many possible meanings

- **All information** on computers is stored as patterns of bits, but…
- How these bits are **interpreted, transformed,** and **displayed** is up to the programmer and the user.



- It means there's **no reason for it to be that way.**
- we just ***know/agree*** that it's how things are.
- One of the biggest things I want you to know is:

What a pattern of bits *means* is arbitrary.

- As a corollary:

The *same* pattern of bits can be *interpreted* many different ways.

The calculat--hum--computer doesn't know or care.

- when writing assembly (and C!) programs, **the computer has no idea what you mean, cause nothing means anything to it**
- "my program assembles/compiles, why is it crashing?"
  - cause the computer is stupid
    - it's a big fast calculator
- there's no difference between nonsense code and useful code
- it's good at doing fun things with bit patterns
- but don't confuse what it does with intelligence
- **every "smart" thing a computer does, it does because a human programmed it to act like that**

Hexadecimal – Base 16

How many symbols in hexadecimal????

16

Shortcomings of binary and decimal

- Binary numbers can get really long, really quickly.
  - **$3{,}927{,}664_{10}$ = 11 1011 1110 1110 0111 0000_2**
- But nice "round" numbers in binary look arbitrary in decimal.
  - **1000000000000000_2 = $32{,}768_{10}$**
- This is because 10 is not a power of 2!
- We could use base-4, base-8, base-16, base-32, etc.
  - Base-4 is not much concise than binary
    - e.g. **$3{,}927{,}664_{10}$ = 120 3331 2323 0000_4**
  - Base-32 and up? would require 32+ symbols. Nope.
    - Well at least for humans… They are actually used!
  - **Base-8** and **base-16** look promising!

Let's make a base-~~2~~ 16 number system

Using base **16**

- A number represented by the digits

$$d_n d_{n-1} \dots d_1 d_0$$

- Has the value

$$d_n \times \mathbf{16}^n + d_{n-1} \times \mathbf{16}^{n-1} + \cdots + d_1 \times \mathbf{16}^1 + d_0 \times \mathbf{16}^0$$

- Using n digits we can represent $\mathbf{16}^n$ different numbers
- The smallest non-negative number representable with n digits is 0
- The largest number representable with n digits is $\mathbf{16}^n - 1$
- Using **16** symbols: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

Hexadecimal, or "hex" (base-16)

- Digit symbols after 9 are A-F, meaning 10-15 respectively.
- Usually we call one hexadecimal digit a *hex digit*. No fancy name :(

$$003B\ EE70 = \begin{aligned} &0 \times 16^7 + \\ &0 \times 16^6 + \\ &3 \times 16^5 + \\ &11 \times 16^4 + \\ &14 \times 16^3 + \\ &14 \times 16^2 + \\ &7 \times 16^1 + \\ &0 \times 16^0 = \end{aligned}$$

$16^7\ \ 16^6\ \ 16^5\ \ 16^4\ \ \ 16^3\ \ 16^2\ \ 16^1\ \ 16^0$

To convert decimal to hex: use a calculator!

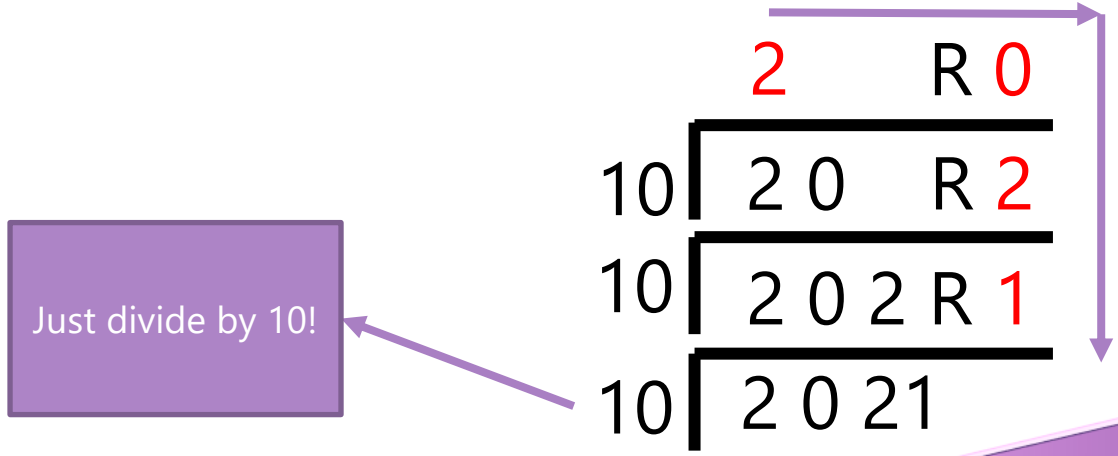
$$3{,}927{,}664_{10}$$

BUT WE REALLY WANT TO KNOW!!

- Ok! then. Let's go back to decimal for a **bit**

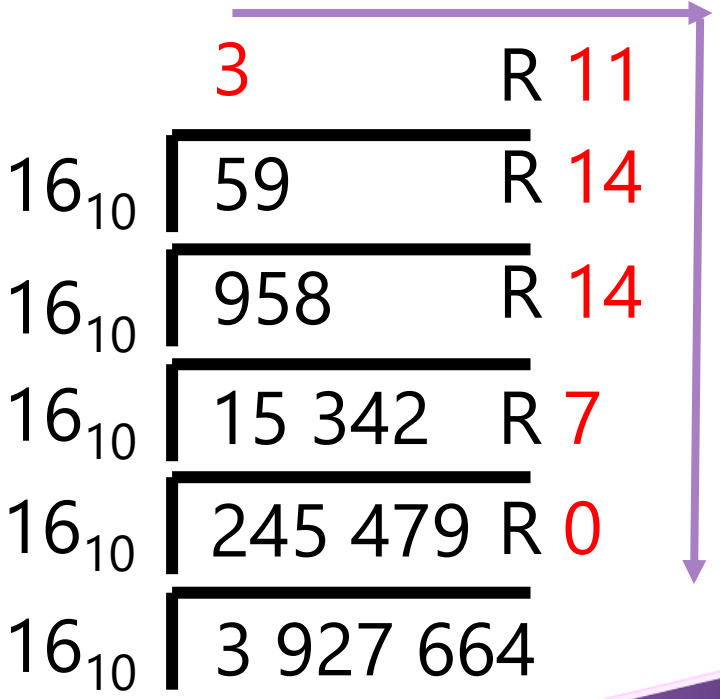
2 0 2 1

How would you extract this number???



- Turns out that dividing by 10 in any base has the same outcome

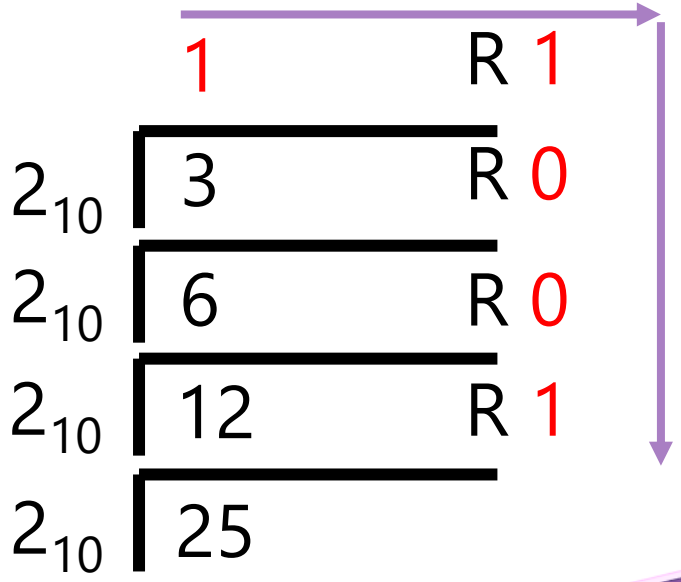
$0x10 \rightarrow 16_{10}$



$003B\ EE70_{16}$

- Turns out that dividing by 10 in any base has the same outcome

$0b10 \rightarrow 2_{10}$



11001_2

The relationship between binary and hex

- **Four bits** are equivalent to **one hex digit.**
- Converting between them is easy!
- Say we had this binary number:

$$\mathbf{111011111011001110000_2}$$

- Starting **from the LSB,** divide into groups of 4 bits (put 0s before the first digits if there are leftovers). Then use the table.

0011 1011 1110 1110 0111 0000

0x3 B E E 7 0

(this is common notation for hex, derived from the C language.)

Bin	Hex	Bin	Hex
0000	0	1000	8
0001	1	1001	9
0010	2	1010	A
0011	3	1011	B
0100	4	1100	C
0101	5	1101	D
0110	6	1110	E
0111	7	1111	F

know how to make this table.

Let's do it!

10010001100111100001101001010101

Let's do it!

00 10	0100	0110	0111	1000	0110	1001	0101
2	4	6	7	8	6	9	5

$$24678695_{16}$$

$$0x24678695$$

1111 | 1111

$$1 \times 2^7 + 1 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

$$1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 4 + 2 + 1 = 15$$

$$(1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0) \times 2^4 + 15$$

Factoring

$2^4 = 16$

$$15 \times 16^1 + 15 \times 16^0$$

F | F

Powers of Two

- Memorize **at least** the powers up to ~2^8 or 2^{10}.
  - If you can't remember one, double the previous one.

- These are the place values for binary, and they are also nice "round" numbers in binary and hex.

- What is the **largest number** that an 8-bit value can hold? What is that in hexadecimal?
  - 255: 0xFF

- How about a 16-bit value?
  - 65535: 0xFFFF

- "0xFFFF" is kinda like "9999" in decimal.

	Dec	Hex
2^0	1	0x1
2^1	2	0x2
2^2	4	0x4
2^3	8	0x8
2^4	16	0x10
2^5	32	0x20
2^6	64	0x40
2^7	128	0x80
2^8	256	0x100

Octal – Base 8

How many symbols in octal????

8

Base-8?

- base-8, **octal**, used to be commonplace but isn't anymore
- each octal digit (0-7) corresponds to *three* bits
  - this made it a nice fit for 9-, 12-, 18-, and 36-bit machines
- buuuut no one cares about octal anymore ☹
- SORRY OCTAL
  - it's okay, it has its revenge from time to time
  - try this out in Java sometime:

```
System.out.println("012345 = " + 012345);
```

Let's do it!

10010001100111100001101001010101

Let's do it!

100	100	011	001	111	000	011	010	010	101
4	4	3	1	7	0	3	2	2	5

$$4431703225_8$$

0 4431703225

0o 4431703225

Next class!

- **Assembly!!!!!**
  - How can we program the computer?
  - New language -> new bugs!

- Specifically
  - Storing and retrieving data from memory
  - Manipulating the data